CPE 486/586: Deep Learning for Engineering Applications

11 Graph Neural Network

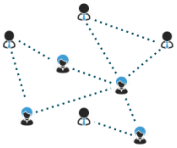
Spring 2026

Rahul Bhadani

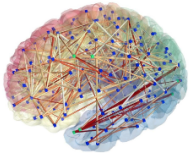
Outline

Motivation

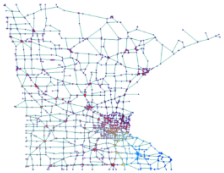
Graph/Network Dataset



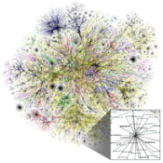
Social networks



Biological networks

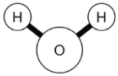


Transportation networks



Communication networks

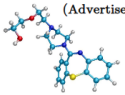
n-body system



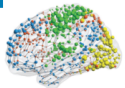
molecule

Graph/Network Dataset

Social networks (Advertisement)



Drug/Material molecules (Chemistry)



Brain connectivity (Neuroscience)

Word relationships (NLP)

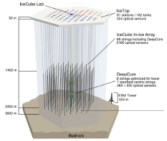
Gene Regulatory Network

3D Meshes (Computer Graphics)

Transportation networks

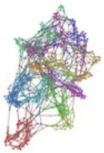
Scene understanding

Recommender systems (Amazon, Netflix)



Neutrino detection (High-energy Physics)

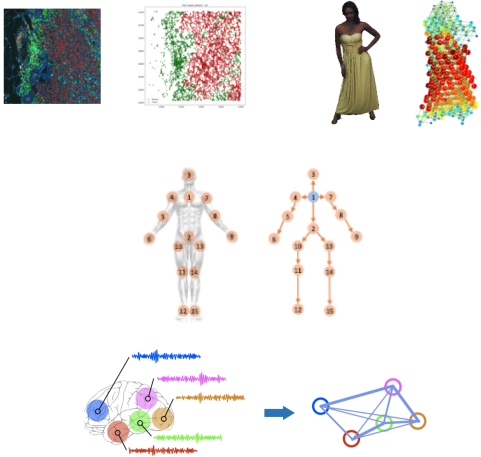
=



Graph

Graphs to Represent Dataset

- Graphs can be used to represent relationships in a structured data.
- They provide a different perspective of dataset.
- Impose relational inductive bias in data
- Lead to knowledge discovery



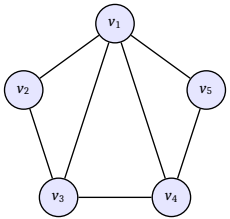
What is Graph?

A graph G is a data structure described by a set of edges and vertices (also known as nodes):

$$G = (V, E) \quad \text{or} \quad G = (V, E, A)$$

- V set of vertices.
- E set of edges.
- They can be directed or undirected.

A graph is often represented by an Adjacency matrix, A. If a graph has N nodes, then A has a dimension of $(N \times N)$. People sometimes provide another feature matrix to describe the nodes in the graph. If each node has F numbers of features, then the feature matrix X has a dimension of $(N \times F)$.



What is Graph?

Graph features:

- Node features: h_i, h_j (e.g. Atom Type)
- Edge features: e_{ij} (e.g. bond type, time to travel from one traffic intersection to another, etc.)
- Overall graph features: g (e.g. molecular energy)

Graphs are non-Euclidean Data

Suitable for data that resides in irregular, non-grid-like structures where relationships are not confined to regular Euclidean spaces.

- Manifolds: Curved surfaces, e.g., 3D shapes or mesh data.
- Point Clouds: Sets of points in 3D space without a grid structure (e.g., LiDAR data).

Real-World Data is Often Non-Euclidean.

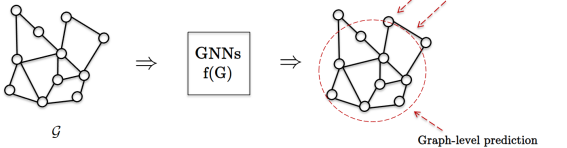
Challenges in handling Non-Euclidean Data

- fixed Grid Assumptions in MLPs, CNNs, RNNs
  - Assume regular, structured data (e.g., grids or sequences).
  - Cannot directly handle irregular neighborhoods or variable node connectivity in graphs or other non-Euclidean structures.
- Irregular Neighborhoods:
  - Varying numbers of neighbors per node.
  - No uniform notion of proximity or direction.
  - Standard convolution filters (which operate on fixed local neighborhoods) fail to adapt to these variable structures.
- Lack of Spatial Regularity:
  - The concept of ”locality” is not fixed and varies across the structure.
  - Non-Euclidean data like point clouds, graphs (undirected) is unordered.

We need a permutation invariance / equivariance.

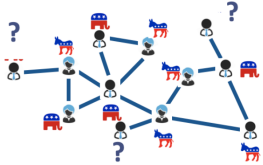
Learning Tasks on Graph-structured Data

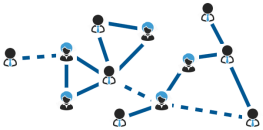
- Graph-wise classification
- Vertex-wise classification/inference of missing values
- Link prediction



is it a 2?
is it a 4?

condition?
no condition?



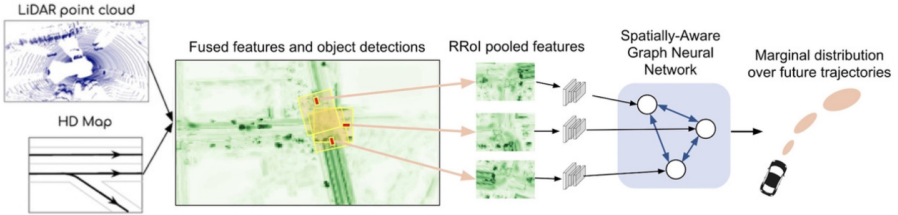


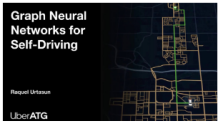
GNN Case Studies

- Chip design (Google)
- Resource management
- Scene reasoning (Meta)
- Recommendation (UberEats/ Pinterest/ Alibaba)
- Fake news detection
- Finance
- Natural Language Processing
- Knowledge graphs (Amazon)
- Transportation (Google Map, Waymo, etc.)
- Autonomous driving (NVIDIA)
- Protein & drug design (Google DeepMind/ Microsoft/ AstraZeneca)
- Energy physics & simulations (Google DeepMind)
- Code bug detection
- Genomics

GNNs for autonomous driving

https://slideslive.com/38930570/graph-neural-networks-for-selfdriving

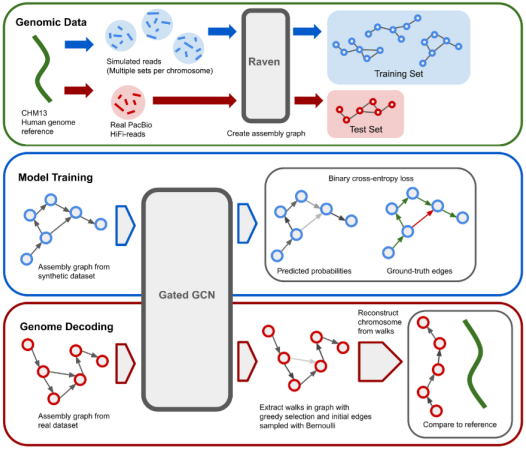




https://slideslive.com/38930570/graph-neural-networks-for-selfdriving

GNNs for Genomics

https://arxiv.org/pdf/2206.00668



Recommended Textbooks

- Graph Representation Learning Book, Springer, 2020
  https://github.com/RHxW/CV-DL-Docs/blob/master/GRL_Book.pdf
- Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges,2021
  https://arxiv.org/pdf/2104.13478
- Graph Neural Networks: Foundations, Frontiers, Applications, Springer,2022
  https://graph-neural-networks.github.io/

GNN Library in PyTorch

- PyTorch Geometric
- Released in March 2019
- https://pytorch-geometric.readthedocs.io/en/latest/

Graph Categories: Natural Graphs

Graphs that are not artificially hand-crafted/constructed but just occur by natural process.

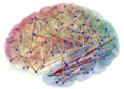
- Social networks : Meta, LinkedIn, Twitter, Tiktok
- Biological networks : Brain connectivity & functionality, gene regulatory networks
- Communication networks : Internet, networking devices
- Transportation networks : Trains, cars, airplanes, pedestrians
- Power networks : Electricity, water


Minnesota Road Network

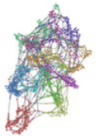
Telecommunication Network


US Electrical Network


Facebook


Brain Connectivity

=


Graphs

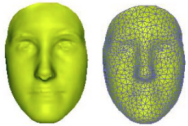
Graph Categories: Explicit Graph Construction from Data

Graphs constructed from data :

- An example can be: compute the correlation netween samples and treating samples as nodes, form an edge if correlation between two sample is greater than some threshold.

GTZAN Music Graph

MNIST Image Graph



3D mesh points

Adjacency Matrix

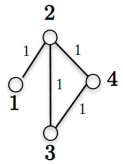
- Matrix A in $G = (V, E, A)$ represents structural/topological information about thenetwork. We have two classes of A.
  - Binary matrix: $A_{ij} \in \{0, 1\}$ and

$$A_{ij} = \begin{cases} 1 & \text{if} \quad (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

  - Weighted matrix: $A_{ij} \in [0, 1]$: commonly normalized to

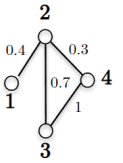
$$A_{ij} = \begin{cases} \in (0, 1] & \text{if} \quad (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

G =



$$\Rightarrow A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

G =



$$\Rightarrow A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 0.4 & 0 & 0 \\ 0.4 & 0 & 0.7 & 0.3 \\ 0 & 0.7 & 0 & 1 \\ 0 & 0.3 & 1 & 0 \end{bmatrix} \end{array}$$

Curse of Dimensionality

In high dimensions, Euclidean distance between data becomes meaningless.

If data are uniformly distributed in $\mathbb{R}^d$,then pick any data point $\mathbf{x}_i \in V$, that gives

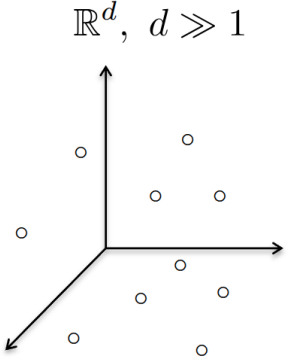
$$\lim_{d\to\infty} \mathbb{E}_{x_i}\left(\frac{d^{\ell_2}_{max}(\mathbf{x}_i, V/\ \mathbf{x}_i) - d^{\ell_2}_{min}(\mathbf{x}_i, V/\ \mathbf{x}_i)}{d^{\ell_2}_{min}(\mathbf{x}_i, V/\ \mathbf{x}_i)}\right) = 0$$

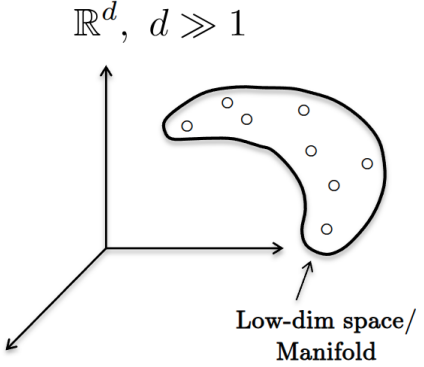
which just means that all data are really far away to each other with the same distance in a very high-dimensional space.

- As an example, there is a loss of intuition in high-dimension.
- You can think that in Normal distribution, in 1-d, most data are concentrated at the center.
- But in high-diemsnional spaces, most data are concentrated on the surface.

But Then There Might Be Some Structure in High-Dimension

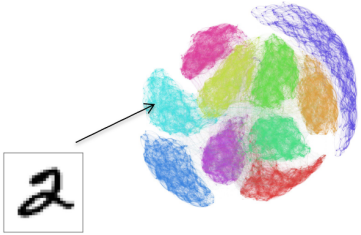
- **Data are uniformly distributed** is actually not true for real-world data.
- Data have always properties, i.e. structures or invariances, such that they belong to a low-dimensional space called manifold where (geodesic) distances are meaningful.



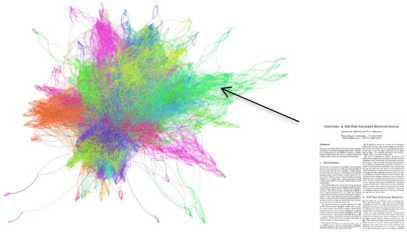


Manifold Learning

A class of algorithms that extracts low-dimensional patterns is manifold learning.



MNIST Image
Graph

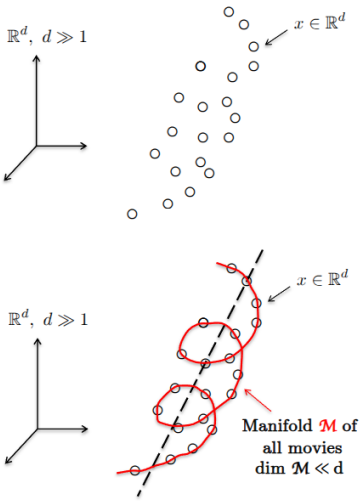


Graph of Text Documents
20newsgroups

Manifolds to Graphs

Primary assumption: High-dimensional data are sampled froma low-dimensional manifold.

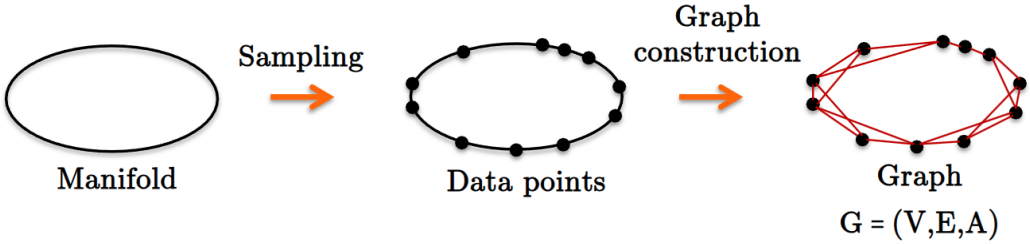
Example: Let x be a movie, each movie is defined by d features/attributes like genre, actors, release year, origin country, etc such that $\mathbf{x}$ in $\mathbb{R}^d$. We can decide to make theassumption that all movies form a manifold $\mathcal{M}$ in $\mathbb{R}^d$.



From Manifolds to Graphs

Graphs can be regarded as a manifold sampling process.

- The manifold information is represented by a neighborhood graph (observe that themanifold is never directly observed or constructed).

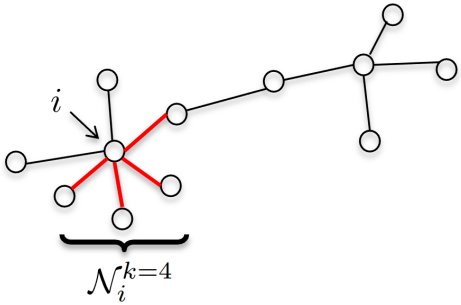


Neighborhood Graphs

- Most populars are $k - NN$ graphs defined as :

$$A_{ij} = \begin{cases} e^{-\frac{dis(\mathbf{x}_i, \mathbf{x}_j)^2}{\sigma^2}} & \text{if } \; j \in \mathcal{N}_i^k \\ 0 & \text{otherwise} \end{cases}$$

where $dist(\mathbf{x}_t, \mathbf{x}_j)$ cane be any kind of distance. $\mathcal{N}_i^k$ is the neighborhood of data $\mathbf{x}_i$.



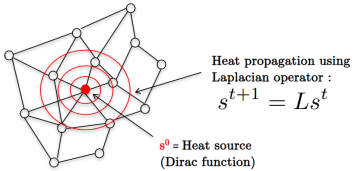
Spectral Graph Theory

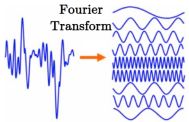
Spectral graph theory (SGT) can

- find meaningful patterns that reveal multi-scale graph structures.
- Process data defined on the graph domain (a.k.a. graph signal processing).
- Boost performance of learning tasks s.a. clustering, classification, recommendation, etc.

Graph Laplacian Operator L.

- Central operator of diffusion processes on graphs.
- Basis functions of this operator are the well-known Fourier modes





Graph Laplacian

- Unnormalized Graph Laplacian:

$$L = D - A$$

where D is the degree matrix : $D = diag(d_1, \cdots, d_n)$, and $d_i i = \sum_j A_{ij}$.

- Normalized Laplacian

$$L_n = D^{-1/2} L D^{-1/2} = I_n - D^{-1/2} A D^{-1/2}$$

- Random Walk Laplacian:

$$L_w = D^{-1} L = I_n - D^{-1} A$$

- All Laplacians are diffusion operators.

Graph Spectrum

Spectral graph theory can extract the modes of variation of the graph system. This can be done using eigenvalue decomposition.

Considering Laplacian operator as L. The Laplacian is symmetric positive semi-definite, so it admits an eigendecomposition:

$$L = U\Lambda U^{\top}$$

where $U = [u_1, u_2, \cdots, u_n]$ are eigenvectors (graph Fourier basis) and $\Lambda = diag(\lambda_1, \lambda_2, \cdots, \lambda_n)$ are eigenvalues with $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$. Also $U^{\top}U = I_n$.

The eigenvectors are the modes of variation. they capture how signals oscillate across the graph topology, from smooth global trends ($\lambda \approx 0$) to high-frequency local variation (λ with a large value).

How to construct graphs from data?

Questions to ask first:

- Which type of graphs?
- Which data distance?
- Which data features?

Optimal Graph Construction

- No universal recipe is available (no theory).
- It depends on data and analysis task (empirical).
- Domain expertise and good practice are essential.

Distance Definition

Euclidean/L2 distance :

- Good distance for low-dimensional data
- Good distance for high-dim data with linearly separable data

Cosine/dot product distance :

- Good distance for high-dim sparse data (.e.g text document)
-

$$d_{cos}(\mathbf{x}_i, \mathbf{x}_j) = \left| \cos^{-1}\left(\frac{\langle \mathbf{x}_i, \mathbf{x}_j \rangle}{||\mathbf{x}_i|| ||\mathbf{x}_j||} \right) \right|$$

Other distances are KL divergence, Wasserstein distance, etc.

Representations for Learning with Graph

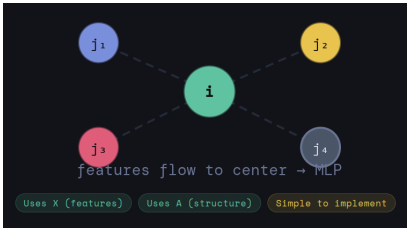
- Each node i carries a feature vector $\mathbf{x}_i$ which gives us $X \in \mathbb{R}^{n \times d}$.
- Adjacency matrix A tells who is conencted to whom.
- Feature matrix X tells what each node knows.

So how do we use X and A to learn something?

Or specifically, how to we prepare their representation that can be fed to a neural network.

Naive Representation

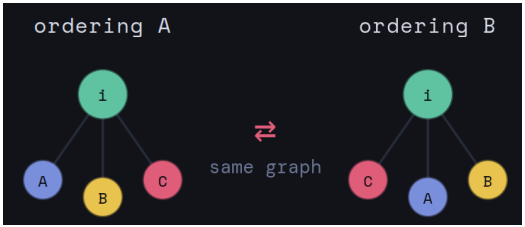
One idea could be to concatenate as $[\mathbf{x}_i$, neighbors of $i]$. Collect all neighbor features, concatenate everything, and feed it to a standard neural network.



Naive Representation

But we have problem!

A graph has no natural ordering of nodes or neighbors. Permuting the nodes gives the same graph, but a completely different input vector to the MLP.



Another issue: different nodes have different numbers of neighbors. An MLP requires a fixed-size input. It cannot natively handle node v with 2 neighbors and node u with 7 neighbors simultaneously.

We need an operation over neighbor features that is independent of ordering and handles any neighborhood size. They are called **A Permutation-Invariant Aggregation**. Example: SUM, MEAN, MAX, etc.

Message Passing Graph Neural Network (Or Spatial GNN)

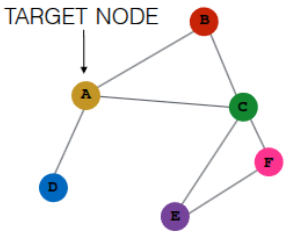
Message Passing:
A mechanism enabling Graph Neural Networks to learn from graph-structured data by having nodes iteratively exchange information with their neighbour's.

- **Message Creation:** Computing messages (information about features) from neighbouring nodes.
- **Message Aggregation:** Aggregating these messages through operations like summation or attention.
- **Node Update:** Updating each node's representation by combining the aggregated information with its current features.

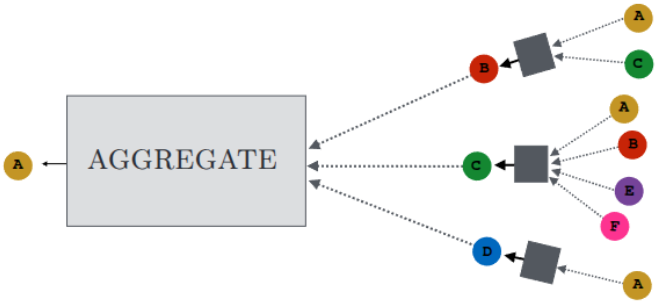
$$\begin{aligned}\mathbf{h}_u^{(k+1)} &= UPDATE^{(k)}\left(\mathbf{h}_u^{(k)}, AGGREGATE^{(k)}(\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\})\right) \\ &= UPDATE^{(k)}\left(\mathbf{h}_u^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right)\end{aligned}$$

where $\mathbf{h}_u^{(k)}$ is the embedding corresponding to the node u, $\mathcal{N}(u)$ is the neighbors of u, $\mathbf{m}$ is the meassage that is being aggregated.
We use superscripts to distinguish the embeddings and functions at different iterations of message passing. In the context of GNN, they are called different **layers** of GNN.

Message Passing Graph Neural Network (Or Spatial GNN)



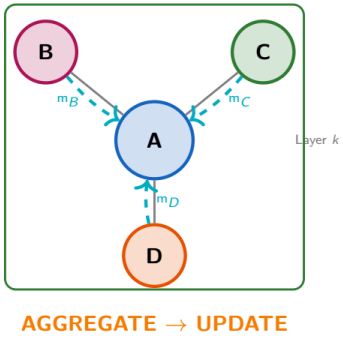
INPUT GRAPH



Message Passing Graph Neural Network (Or Spatial GNN)

Initial embeddings are set as $\mathbf{h}_u^{(0)} = \mathbf{x}_u \forall u \in Vcal$. After running k iteration, we can use the output of the final layer to define the embeddings of each node:

$$\mathbf{z}_u = \mathbf{h}_u^k, \quad \forall u \in \mathcal{V}$$



Note: Since the AGGREGATE function takes a set as input, GNNs defined in this way are permutation equivariant by design.

The Most Basic GNN

Basic GNN Message Passing

$$\mathbf{h}_u^{(k)} = \sigma\Big(\mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\Big)$$

Many GNNs can also be succinctly defined using graph-level equations. In the case of a basic GNN, we can write the graph-level definition of the model as follows:

Graph-level Form

$$H^{(t)} = \sigma\Big(AH^{(t-1)}\mathbf{W}_{\text{neigh}}^{(t)} + H^{(t-1)}\mathbf{W}_{\text{self}}^{(t)}\Big)$$

Efficient via **sparse matrix operations**

$\mathbf{H}^{(t)} \in \mathbb{R}^{|V| \times d}$ denotes the matrix of node representations at layer t in the GNN. $\mathbf{A}$ is the graph adjacency matrix. This one omits bias for simplicity.

The Most Basic GNN: An Abstract Form

$$\mathbf{x}_v^{(k+1)} = f_\theta^{(k+1)}\left(\mathbf{x}_v^{(k)}, \left\{\mathbf{x}_w^{(k)} : w \in \mathcal{N}(v)\right\}\right)$$

where node features $\mathbf{x}_v^{(k)}$ of all nodes $v \in \mathcal{V}$ in a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ are iteratively updated by aggregating localized information from their neighbors $\mathcal{N}(v)$.

Neighborhood Normalization

Problem: High-degree nodes produce large $\| \sum_v \mathbf{h}_v \|$, causing *instability*.

Mean Normalization

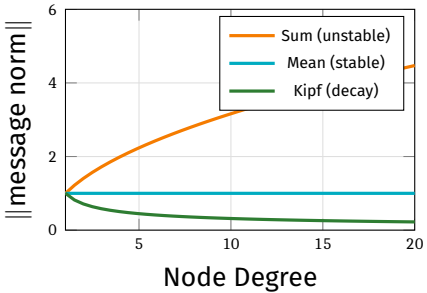
$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h}_v}{|\mathcal{N}(u)|}$$

Kipf Normalization (GCN)

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}$$

Down-weights high-degree neighbors

Effect of Normalization



Note

Normalization **trades expressivity for stability**. The mean aggregator is provably less powerful than the sum aggregator.

Graph Convolutional Network (GCN)

GCN Layer (Kipf & Welling, 2016)

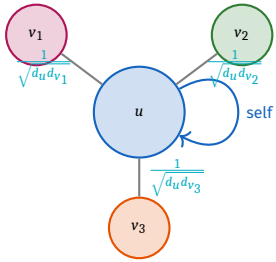
$$\mathbf{h}_v^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}\right)$$

Combines: **Kipf normalization** + **self-loop** trick

$$H^{(t)} = \sigma\Big((A + I)H^{(t-1)}W^{(t)}\Big)$$

Properties

✓ Permutation equivariant
✓ Efficient sparse ops
✓ First-order spectral approximation



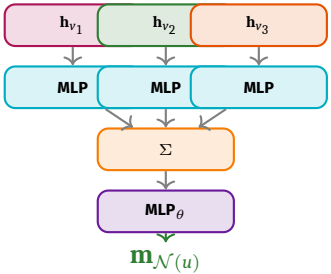
GCN aggregation weights

Set Aggregators: Going Beyond Sum/Mean

Set Pooling (Zaheer et al., 2017)

Universal approximator for permutation-invariant functions:

$$\mathbf{m}_{\mathcal{N}(u)} = \text{MLP}_\theta\left(\sum_{v \in \mathcal{N}(u)} \text{MLP}(\mathbf{h}_v)\right)$$



Set Pooling architecture

Janossy Pooling (Murphy et al., 2018)

Apply a *sequence model* on permutations, then average:

$$\mathbf{m}_{\mathcal{N}(u)} = \text{MLP}_\theta\left(\frac{1}{|\Pi|}\sum_{\pi \in \Pi} \rho(\{\mathbf{h}_v\}_\pi)\right)$$

where ρ is typically an **LSTM**.

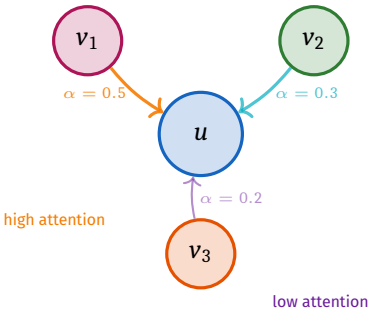
Graph Attention Networks (GAT)

Attention-based Aggregation

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v}\, \mathbf{h}_v$$

where the attention weight is:

$$\alpha_{u,v} = \frac{\exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \| \mathbf{W}\mathbf{h}_v]\right)}{\sum_{v' \in \mathcal{N}(u)} \exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \| \mathbf{W}\mathbf{h}'_v]\right)}$$



Multi-head Attention

$$\mathbf{m}_{\mathcal{N}(u)} = [\mathbf{a}_1 \| \mathbf{a}_2 \| \cdots \| \mathbf{a}_K]$$

$$\mathbf{a}_k = W_k \sum_{v \in \mathcal{N}(u)} \alpha_{u,v,k}\, \mathbf{h}_v$$

Connection to Transformers

A Transformer layer $\equiv$ a GAT on a **fully-connected** graph.
Complexity: $\mathcal{O}(|V|^2)$ vs $\mathcal{O}(|V||E|)$ for sparse GNN.

Over-Smoothing: A Core Challenge

Over-Smoothing

After many layers, **all node embeddings converge** to nearly the same vector – local neighborhood information is lost.

Influence Score (Xu et al., 2018):

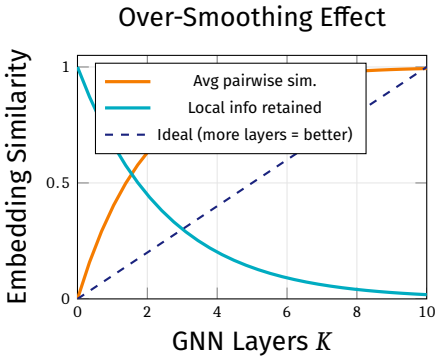
$$I_K(u, v) = \mathbf{1}^\top \left(\frac{\partial \mathbf{h}_v^{(K)}}{\partial \mathbf{h}_u^{(0)}} \right) \mathbf{1}$$

Theorem (Xu et al.)

For GCN-style models with normalized aggregation:

$$I_K(u, v) \propto p_{G,K}(v \mid u)$$

the K-step random walk probability from u to v.



Adding more layers can **hurt** performance!

Addressing Over-Smoothing: Skip Connections

Concatenation Skip (GraphSAGE)

$$\text{UPDATE}_{\text{concat}}(\mathbf{h}_u, \mathbf{m}) = \left[\text{UPDATE}_{\text{base}}(\mathbf{h}_u, \mathbf{m}) \parallel \mathbf{h}_u\right]$$

Explicitly preserves previous-layer info.

Gated Updates (GRU/LSTM)

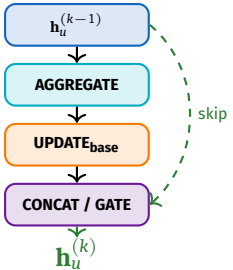
$$\mathbf{h}_u^{(k)} = \text{GRU}\left(\mathbf{h}_u^{(k-1)},\ \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right)$$

Parameters **shared across nodes and layers**.

Gated Interpolation (Pham et al., 2017)

$$\text{UPDATE}_{\text{interp}} = \boldsymbol{\alpha}_1 \odot \text{UPDATE}_{\text{base}} + \boldsymbol{\alpha}_2 \odot \mathbf{h}_u$$

$\boldsymbol{\alpha}_1 + \boldsymbol{\alpha}_2 = \mathbf{1}$; gates learned from data.



Skip connection architecture

Jumping Knowledge Connections

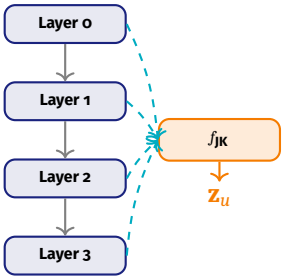
JK Networks (Xu et al., 2018)

Instead of only using the final layer, **combine all layers**:

$$\mathbf{z}_u = f_{\text{JK}}\big(\mathbf{h}_u^{(0)} \| \mathbf{h}_u^{(1)} \| \cdots \| \mathbf{h}_u^{(K)}\big)$$

f_{JK} can be:

- **Concatenation** (identity)
- Max-pooling over layers
- LSTM attention over layers



Benefit

Each node can **adaptively select** the right neighborhood size – some nodes need local info, others global.

Relational Graph Convolutional Networks (R-GCN)

Challenge: Graphs with *multiple edge types* (e.g., knowledge graphs).

RGCN Aggregation (Schlichtkrull et al., 2017)

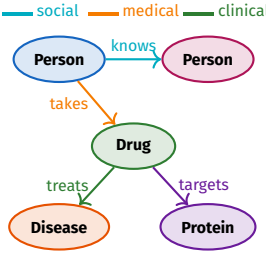
$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{\tau \in \mathcal{R}} \sum_{v \in \mathcal{N}_\tau(u)} \frac{W_\tau \, \mathbf{h}_v}{f_n(\mathcal{N}(u), \mathcal{N}(v))}$$

One trainable matrix W_τ **per relation type**.

Parameter Sharing via Basis Decomposition

$$W_\tau = \sum_{b=1}^{B} \alpha_{b,\tau} \, \mathbf{B}_b$$

Shared basis matrices $\mathbf{B}_1, \ldots, \mathbf{B}_B$ across all relations.



Knowledge graph with multiple relation types

Graph-Level Embeddings: Pooling Strategies

Goal: $\{z_1, \ldots, z_{|V|}\} \longrightarrow z_G$

Set Pooling

Simple sum / mean:

$$z_G = \frac{\sum_{v \in V} z_v}{f_n(|V|)}$$

Effective for **small graphs** (e.g., molecules).

Attention Pooling (Vinyals et al., 2015)

Iterative attention over nodes:

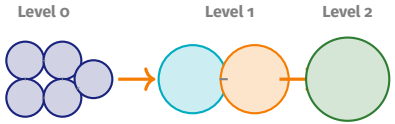
$$q_t = \text{LSTM}(o_{t-1}, q_{t-1}), \quad a_{v,t} \propto \exp(f_a(z_v, q_t))$$
$$o_t = \sum_v a_{v,t} z_v$$

$$z_G = o_1 \| o_2 \| \cdots \| o_T$$

Graph Coarsening (DiffPool)

Hierarchically collapse nodes into clusters:

$$S = f_c(G, Z) \in \mathbb{R}^{|V| \times c}$$
$$A_{\text{new}} = S^\top A S$$
$$X_{\text{new}} = S^\top Z$$



Hierarchical graph coarsening

Additional Reading

- https://tkipf.github.io/graph-convolutional-networks/
- https://distill.pub/2021/gnn-intro/
- https://medium.com/@rajeev.chandran_61731/understanding-message-passing-frameworks-in-graph-neural-networks-944d9e2a1105
- The Graph Neural Network Model https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_mar29.pdf

Use Case

https://docs.google.com/presentation/d/1psyAe3x8A67eM0eauX34Ii9igefC1124/edit?usp=sharing&ouid=102329043416263329658&rtpof=true&sd=true